

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

Лабораторная работа 2

по дисциплине **Программное обеспечение мехатронных и робототехнических систем**

Козел З. М. _____
Студенты группы: КВБО-07-23 Болдырева А. А. _____
Абрамов А. С. _____
Преподаватель: Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

Программное обеспечение системы управления мехатронного модуля управления защитной дверью

Цель работы: получение навыков построения программного обеспечения промышленных систем управления на базе функциональных блоков и конечных автоматов.

Задание: разработать программное обеспечение системы автоматического управления приводом защитной двери. Схема механизма представлена на рисунке 1. Дверь оснащена асинхронным двигателем и четырьмя датчиками положения (S0 – S3), которые реагируют на пластину, закреплённую на двери. Открытие и закрытие двери управляется тумблером.

При включении системы управления дверь должна двигаться в заданную сторону на небольшой скорости для определения своего местоположения, при этом индикаторы должны мерцать с частотой 1 Гц. После этого происходит переход в рабочий режим. В рабочем режиме необходимо обеспечить максимально возможную скорость движения на отрезке S1–S2 и небольшую скорость на отрезках S0–S1 и S2–S3 (для безопасности движения). Индикаторы должны показывать местоположение двери. Система управления должна быть выполнена в виде функционального блока, предполагающего повторное использование.

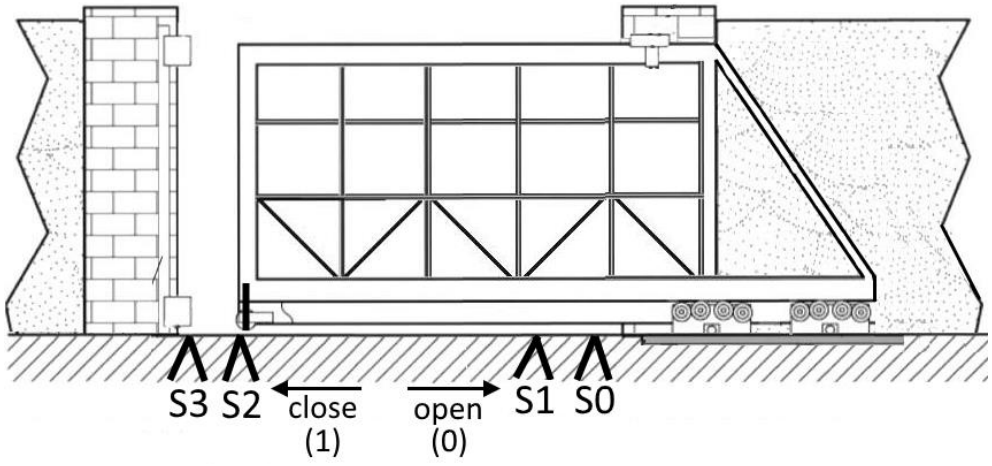


Рисунок 1. Устройство защитной двери

Ход работы

1. Конфигурация аппаратной части стенда и настройка частотного преобразователя

Лабораторный стенд построен на базе панельного программируемого логического контроллера (ПЛК) B&R 4PP065_0571_P74. По внутренней шине X2X Link к контроллеру подключены модули ввода-вывода серии X20, среди которых в данной работе задействованы модуль дискретного ввода X20DI9371 (принимает сигналы от тумблера направления и четырех кнопок, имитирующих датчики S0–S3) и модуль дискретного вывода X20DO9322 (управляет четырьмя светодиодными индикаторами). Управление асинхронным двигателем осуществляется через частотный преобразователь ACOPOSinverter.

В среде B&R Automation Studio был распакован исходный проект стенда, после чего в графическом редакторе аппаратной конфигурации System Designer (Hardware.hwl) к крайнему модулю шины X2X был добавлен и подключен частотный преобразователь 8I64xxxxxxx.00x-1 (рисунок 2).

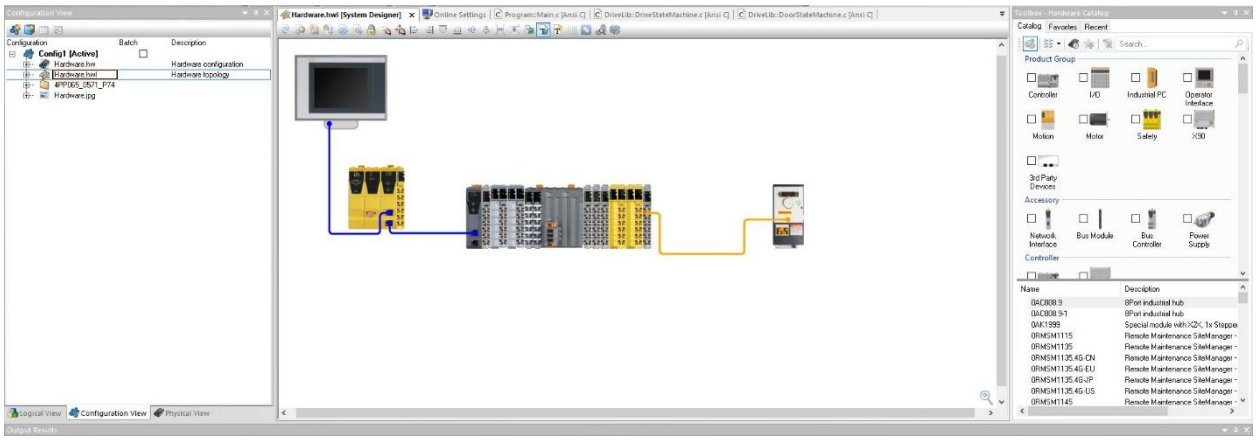


Рисунок 2. Аппаратная топология стенда с подключенным частотным преобразователем

Для согласования частотного преобразователя с реальным двигателем в окне Physical View через пункт контекстного меню «Configuration» модуля 8I64xxxxxxx.00x-1 были внесены паспортные параметры асинхронного электродвигателя (раздел DRC – Motor control menu), а также проверены настройки времени разгона, торможения и предельной частоты (раздел SET –

Settings menu). Установленные значения сведены в таблицу 1 и показаны на рисунке 3.

Таблица 1. Паспортные параметры асинхронного двигателя и настройки частотного преобразователя

Наименование параметра	Параметр в конфигурации	Значение	Ед. изм.
Номинальное питающее напряжение двигателя	UNS Rated motor volt.	220	В
Номинальная частота питающей сети	FRS Rated motor freq.	500	0,1 Гц
Сопротивление статора в холодном состоянии	RSC Cold stator resist.	33000	мОм
Коэффициент мощности (cos φ)	COS Motor 1 Cosinus Phi	64	0,01
Номинальная частота вращения вала	NSP Rated motor speed	890	об/мин
Номинальный ток статора	NCR Rated mot. current	10	0,1 А
Время разгона до номинальной частоты	ACC Acceleration	30	0,1 с
Время торможения до полной остановки	DEC Deceleration	30	0,1 с
Верхний предел выходной частоты	HSP High Speed	500	0,1 Гц

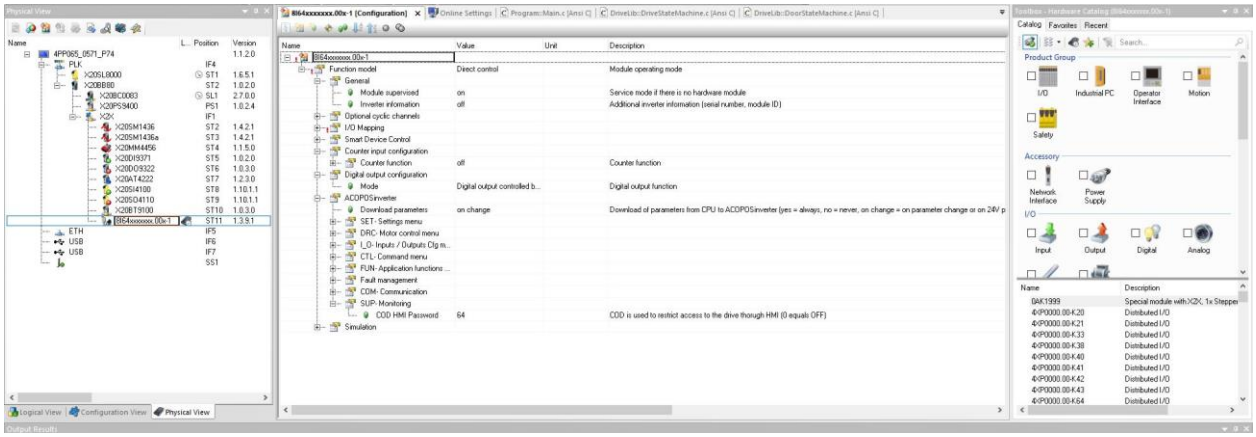


Рисунок 3. Настройка параметров двигателя и инвертора в окне Configuration

По умолчанию регистры состояния (ETAD, ETI) и управления (CMDD, CMI) инвертора представлены в конфигурации отдельными битовыми каналами (unpacked), что потребовало бы создания десятков булевых переменных. Для упрощения обмена данными в разделе «I/O Mapping» конфигурации преобразователя формат представления этих четырех регистров был изменен на упакованный — «packed» (рисунок 4). В результате

каждое слово состояния и управления стало доступно как единый 16-битный целочисленный канал типа UINT (ETAD_Input и CMDD_Output).

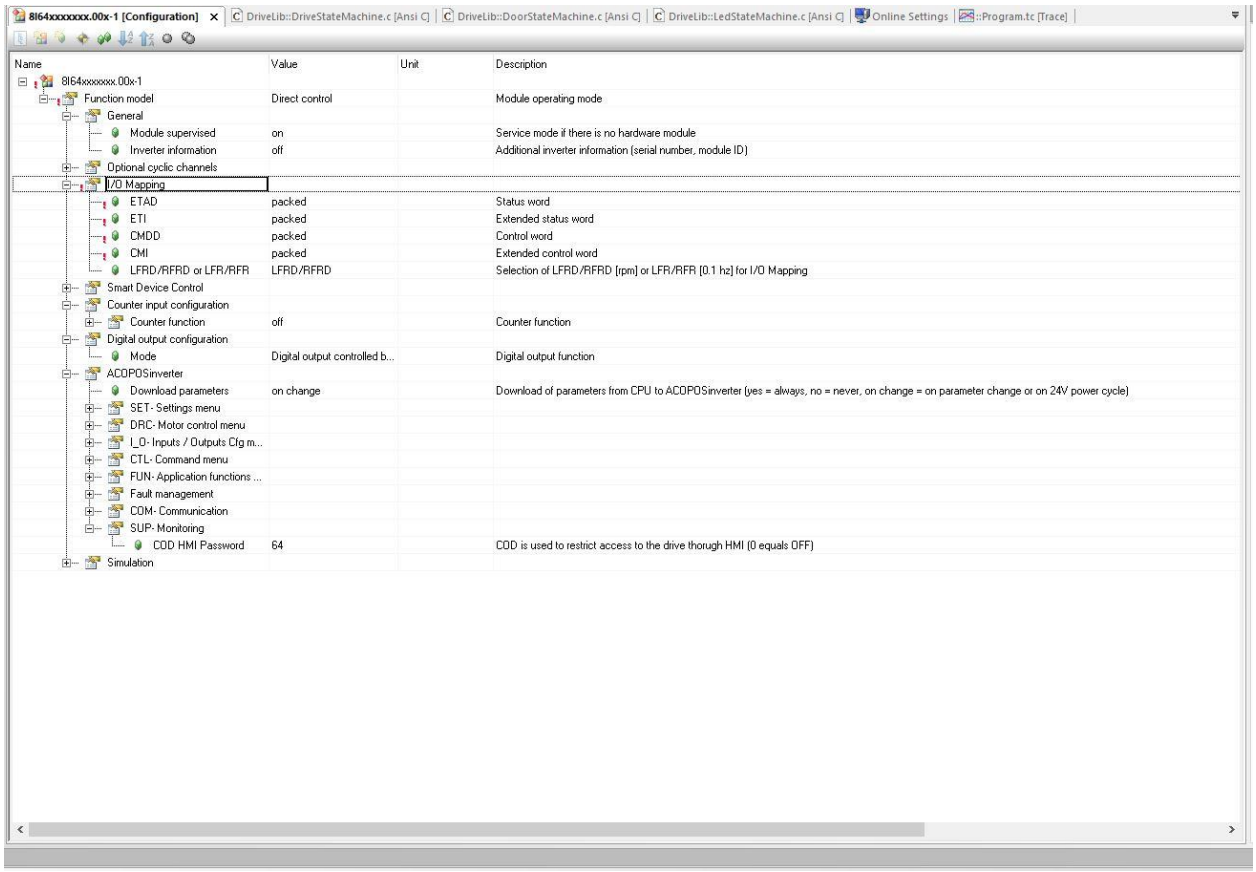


Рисунок 4. Перевод статусных и управляющих регистров инвертора в режим packed

2. Разработка программных компонентов и библиотеки DriveLib

В дереве проекта Logical View были созданы основная программа Program на языке ANSI C (файл Main.c, назначенный в циклический класс задач Cyclic#4 с периодом вызова 100 мс) и пользовательская библиотека DriveLib (рисунок 5). Вся прикладная логика разделена на три функциональных блока внутри библиотеки DriveLib:

– **DriveStateMachine —** отвечает за инициализацию частотного преобразователя и перевод его силовой части в рабочее состояние по слову статуса;

– **DoorStateMachine —** реализует конечный автомат управления дверью (определение текущего состояния и задание скорости вращения привода по сигналам датчиков и тумблера);

– **LedStateMachine** — управляет четырьмя светодиодными индикаторами в зависимости от текущего состояния двери.

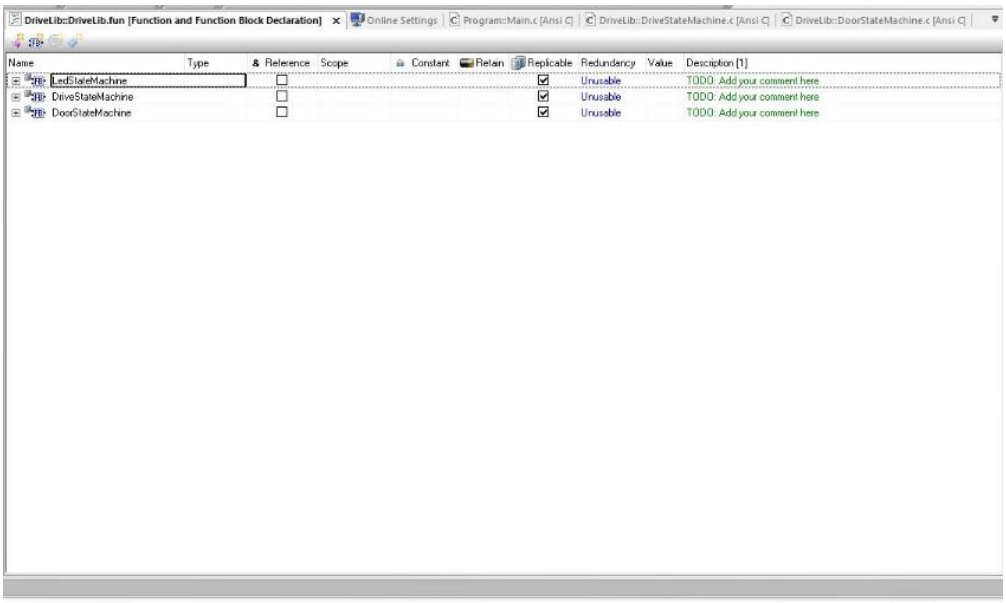


Рисунок 5. Состав функциональных блоков библиотеки DriveLib

Декларация входных и выходных переменных всех трех функциональных блоков в файле DriveLib.fun показана на рисунке 6, а их подробное описание приведено в сводной таблице 2.

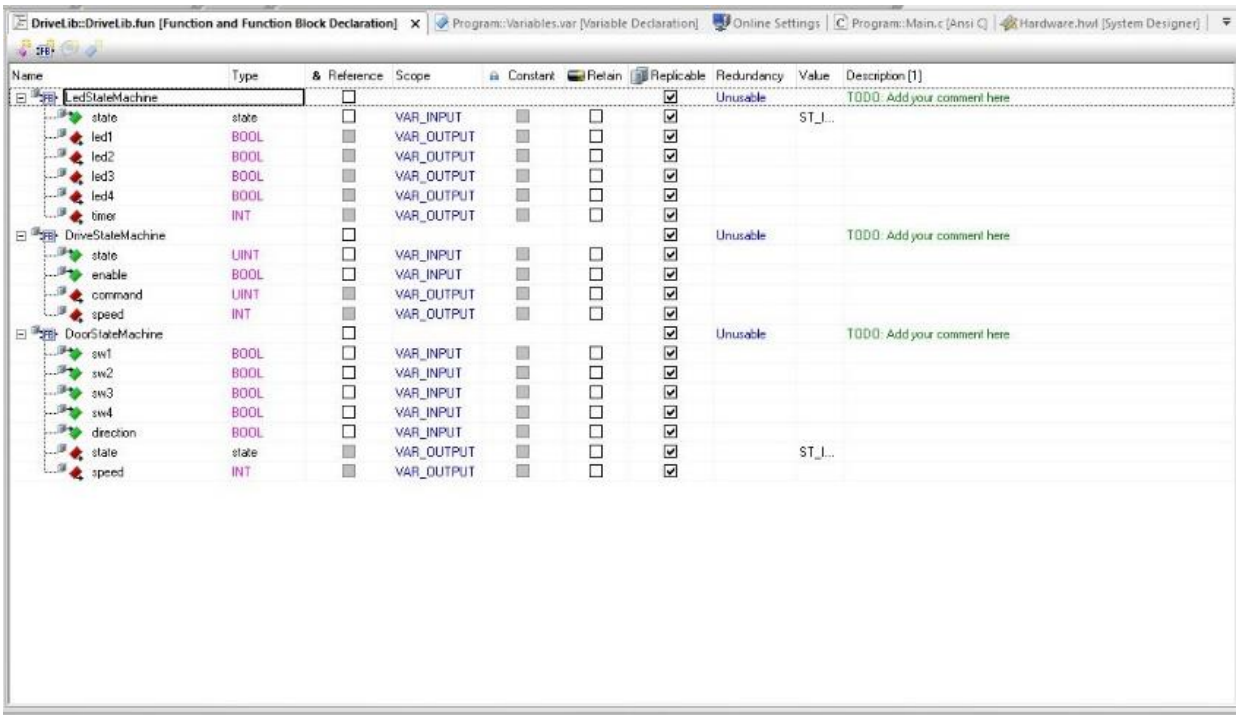


Рисунок 6. Объявление структуры функциональных блоков в файле DriveLib.fun

Таблица 2. Входные и выходные параметры функциональных блоков библиотеки DriveLib

Функциональный блок	Направление (Scope)	Имя поля	Тип данных	Назначение переменной
DriveStateMachine	Вход (VAR_INPUT)	state	UINT	Слово состояния частотного преобразователя (ETAD)
DriveStateMachine	Вход (VAR_INPUT)	enable	BOOL	Сигнал разрешения работы привода
DriveStateMachine	Выход (VAR_OUTPUT)	command	UINT	Командное слово управления инвертором (CMDD)
DriveStateMachine	Выход (VAR_OUTPUT)	speed	INT	Заданная скорость вращения вала, об/мин (LFRD)
DoorStateMachine	Вход (VAR_INPUT)	sw1 .. sw4	BOOL	Сигналы путевых датчиков положения S0, S1, S2, S3
DoorStateMachine	Вход (VAR_INPUT)	direction	BOOL	Команда тумблера направления (0 — откр., 1 — закр.)
DoorStateMachine	Выход (VAR_OUTPUT)	state	state	Текущее состояние автомата двери (исходное ST_INIT)
DoorStateMachine	Выход (VAR_OUTPUT)	speed	INT	Рассчитанная уставка скорости двери, об/мин
LedStateMachine	Вход (VAR_INPUT)	state	state	Текущее состояние двери (от блока DoorStateMachine)
LedStateMachine	Выход (VAR_OUTPUT)	led1 .. led4	BOOL	Выходные сигналы управления светодиодами 1–4
LedStateMachine	Выход (VAR_OUTPUT)	timer	INT	Счетчик циклов задачи для формирования мигания 1 Гц

3. Описание и реализация конечных автоматов

3.1. Автомат инициализации частотного преобразователя (DriveStateMachine)

Для запуска асинхронного привода частотный преобразователь должен пройти обязательную последовательность состояний: из состояния запрета включения (DISABLED) через состояние готовности силовой цепи (SWITCHED_ON) в рабочее состояние (READY). В блоке DriveStateMachine на входное слово состояния state накладывается битовая маска 0x6F (UINT mask = inst->state & 0x6f;), позволяющая выделить статусные биты привода. Если вход разрешения enable равен 0, на инвертор принудительно подается

команда отключения CMD_SHUTDOWN. При enable = 1 блок формирует командное слово command согласно таблице 3 и блок-схеме на рисунке 8.

Таблица 3. Константы библиотеки DriveLib и выбор команды управления инвертором

Условие (enable и state & 0x6F)	Константа состояния	Код	Выдаваемая команда (command)	Код команды
enable = 0 (в любом состоянии)	—	—	CMD_SHUTDOWN (Отключение привода)	6
enable = 1, state & 0x6F == 64	STATE_DISABLED	64	CMD_SHUTDOWN (Переход к готовности)	6
enable = 1, state & 0x6F == 35	STATE_SWITCHED_ON	35	CMD_SWITCH_ON (Включение питания)	7
enable = 1, state & 0x6F == 33	STATE_READY	33	CMD_ENABLE (Разрешение вращения)	15

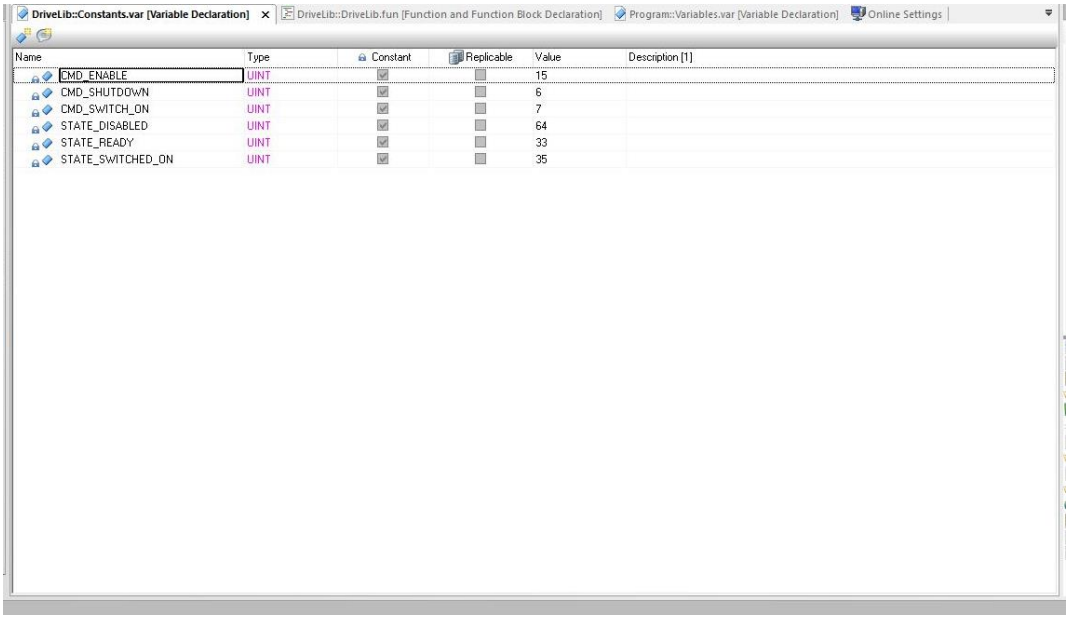


Рисунок 7. Объявление именованных констант в файле Constants.var

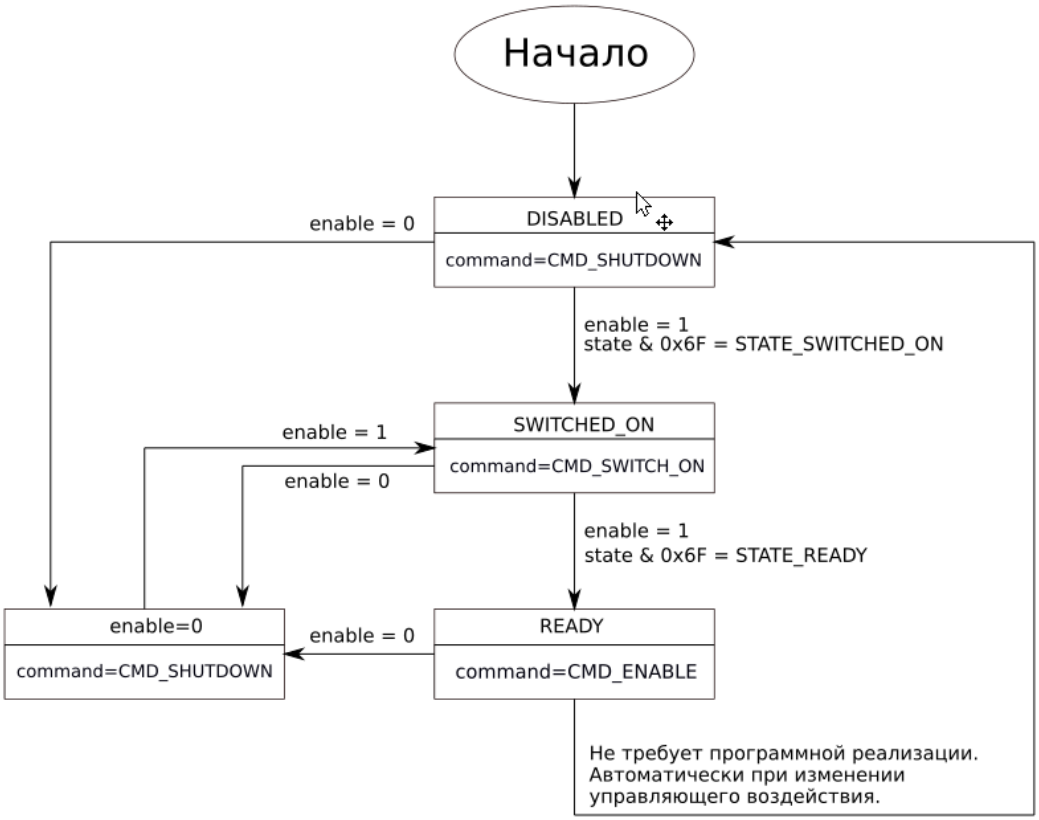
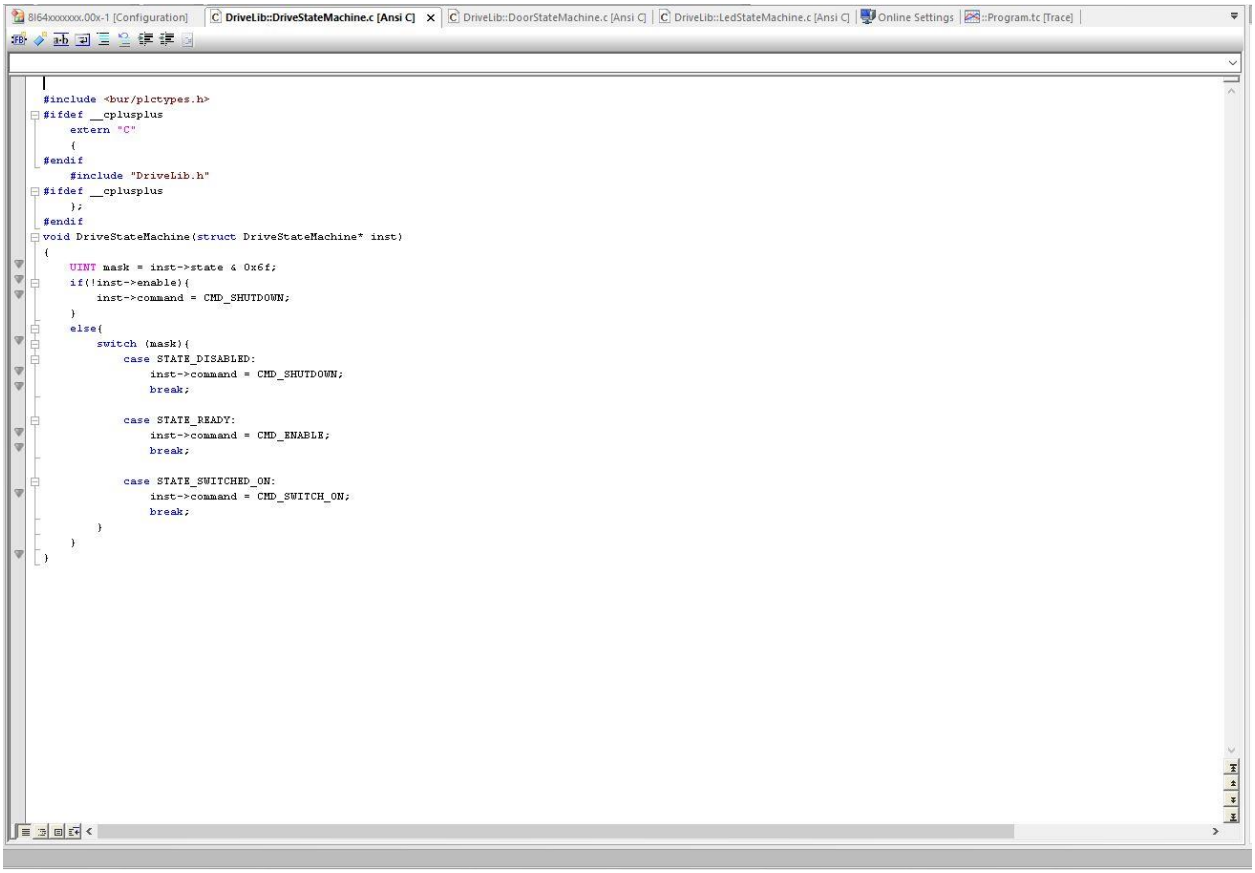


Рисунок 8. Блок-схема автомата управления частотным преобразователем



```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "DriveLib.h"
#ifdef __cplusplus
    };
#endif
void DriveStateMachine(struct DriveStateMachine* inst)
{
    UINT mask = inst->state & 0x6f;
    if(!inst->enable){
        inst->command = CMD_SHUTDOWN;
    }
    else{
        switch (mask){
            case STATE_DISABLED:
                inst->command = CMD_SHUTDOWN;
                break;

            case STATE_READY:
                inst->command = CMD_ENABLE;
                break;

            case STATE_SWITCHED_ON:
                inst->command = CMD_SWITCH_ON;
                break;
        }
    }
}
```

Рисунок 9. Программная реализация блока DriveStateMachine на языке ANSI C

3.2. Конечный автомат управления дверью (DoorStateMachine)

Логика движения двери включает 10 состояний, которые оформлены в файле Types.typ в виде пользовательского перечисления state с кодами от 0 до 9 (рисунок 10). В программе принято соглашение, согласующее физическое расположение тумблера, кнопок-датчиков и индикаторов на стенде: открытому положению двери соответствует левая крайняя точка (датчик S0, вход sw1), а закрытому — правая крайняя точка (датчик S3, вход sw4). При положении тумблера direction = 0 задается движение на открытие (отрицательная скорость), а при direction = 1 — движение на закрытие (положительная скорость).

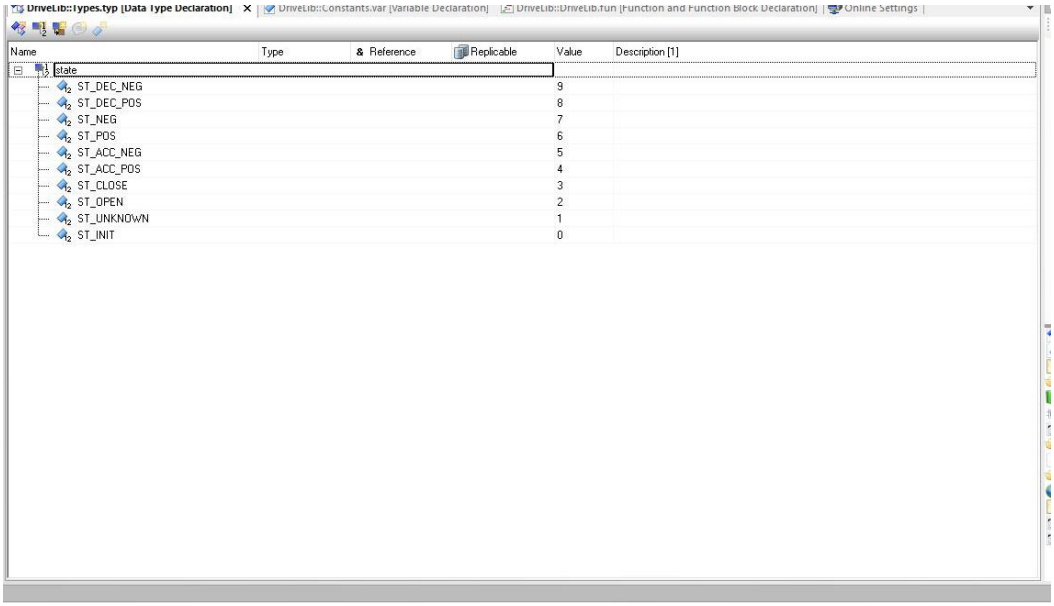


Рисунок 10. Перечисление состояний двери state в файле Types.typ

Граф состояний и переходов автомата представлен на рисунке 11, а полная таблица переходов, задаваемых скоростей и кодов светодиодной индикации приведена в таблице 4.

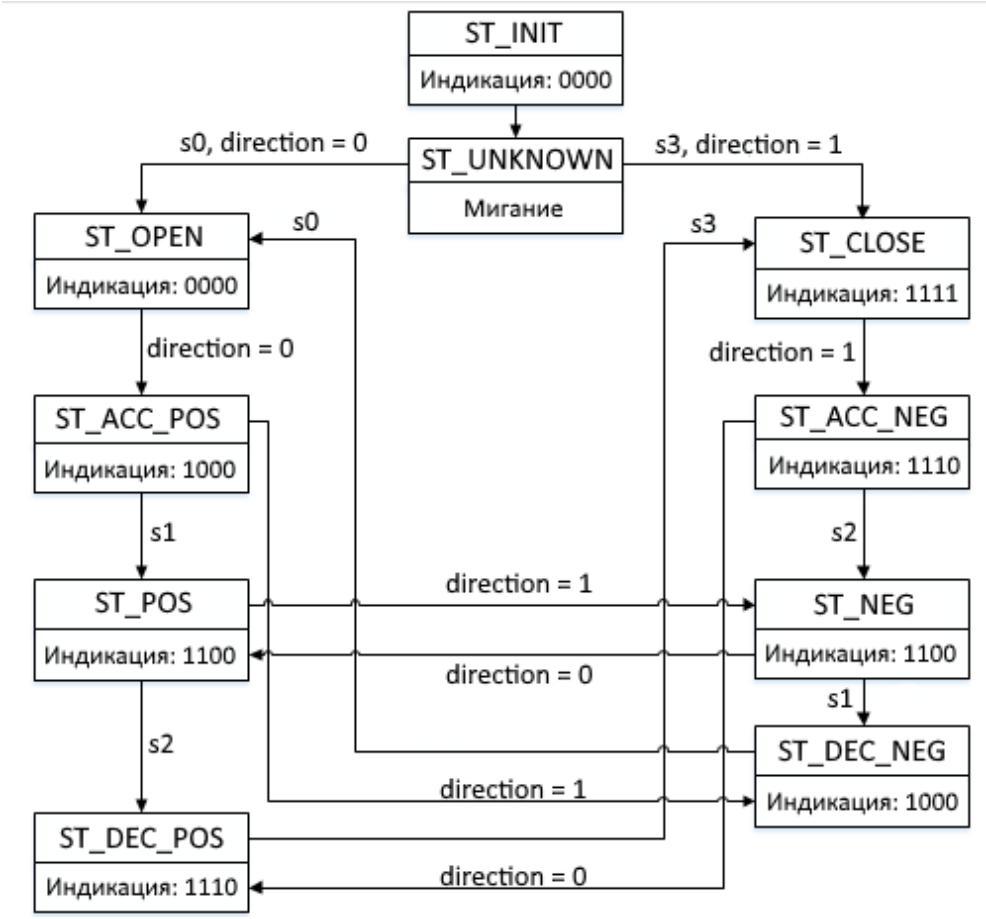


Рисунок 11. Блок-схема состояний защитной двери и переходов между ними

Таблица 4. Описание состояний, скоростей, индикации и условий переходов автомата DoorStateMachine

Состояние (код)	Режим работы двери	Скорость, об/мин	Индикация (led1..4)	Условия перехода в следующее состояние
ST_INIT (0)	Инициализация при запуске	0	0000	Безусловный переход в ST_UNKNOWN
ST_UNKNOWN (1)	Поиск положения на малой скорости	−50 (dir=0) +50 (dir=1)	Мигание 1 Гц	• sw1=1 и dir=0 → ST_OPEN • sw4=1 и dir=1 → ST_CLOSE
ST_OPEN (2)	Дверь открыта (у датчика S0)	0	0000	• dir=1 → ST_ACC_POS • dir=0 → ожидание (speed = 0)
ST_ACC_POS (4)	Разгон на закрытие (участок S0–S1)	+100	1000	• dir=1 и sw2=1 (S1) → ST_POS • dir=0 (реверс) → ST_DEC_NEG
ST_POS (6)	Быстрое закрытие (участок S1–S2)	+200	1100	• dir=1 и sw3=1 (S2) → ST_DEC_POS • dir=0 (реверс) → ST_NEG
ST_DEC_POS (8)	Замедление перед закрытием (S2–S3)	+100	1110	• dir=1 и sw4=1 (S3) → ST_CLOSE • dir=0 (реверс) → ST_ACC_NEG

Состояние (код)	Режим работы двери	Скорость, об/мин	Индикация (led1..4)	Условия перехода в следующее состояние
ST_CLOSE (3)	Дверь закрыта (у датчика S3)	0	1111	• dir=0 → ST_ACC_NEG • dir=1 → ожидание (speed = 0)
ST_ACC_NEG (5)	Разгон на открытие (участок S3–S2)	−100	1110	• sw3=1 (S2) → ST_NEG • dir=1 и sw3=0 (реверс) → ST_DEC_POS
ST_NEG (7)	Быстрое открытие (участок S2–S1)	−200	1100	• dir=0 и sw2=1 (S1) → ST_DEC_NEG • dir=1 (реверс) → ST_POS
ST_DEC_NEG (9)	Замедление перед открытием (S1–S0)	−100	1000	• dir=0 и sw1=1 (S0) → ST_OPEN • dir=1 (реверс) → ST_ACC_POS

Как следует из таблицы 4, максимальная скорость (±200 об/мин) развивается исключительно на безопасном центральном участке S1–S2, тогда как при трогании с места и при подходе к концевым упорам скорость снижена вдвое (±100 об/мин). При переключении тумблера во время движения автомат без остановки переходит в парное состояние обратного хода для того же участка пути (ST_ACC_POS ↔ ST_DEC_NEG, ST_POS ↔ ST_NEG, ST_DEC_POS ↔ ST_ACC_NEG).

3.3. Автомат управления светодиодной индикацией (LedStateMachine)

Функциональный блок LedStateMachine принимает текущее состояние двери state и включает светодиоды led1–led4 в соответствии с таблицей 4: по мере закрытия двери число горящих индикаторов увеличивается от 0000 (открыта) до 1111 (закрыта), а при открытии — уменьшается в обратном порядке.

В состоянии начального поиска ST_UNKNOWN все четыре индикатора должны мигать с частотой 1 Гц (период T = 1 с, то есть переключение каждые 0,5 с). Для этого в локальных переменных основной программы объявлен счетчик timeCounter типа USINT (рисунок 12), который инкрементируется в каждом цикле задачи ProgramCyclic и передается на вход ledSM.timer. Поскольку период вызова класса задач Cyclic#4 равен 100 мс, полупериоду 0,5

с соответствуют ровно 5 тактов. Внутри блока LedStateMachine проверяется условие кратности пяти (if (inst->timer % 5 == 0)), при выполнении которого состояния всех четырех светодиодов инвертируются.

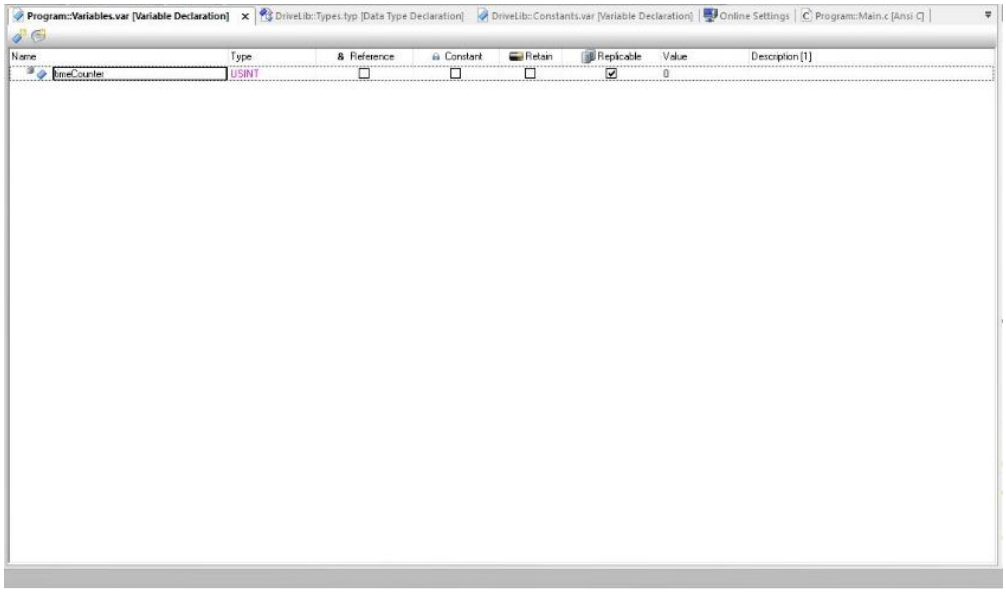


Рисунок 12. Объявление локальной переменной-счетчика timeCounter в Variables.var

4. Привязка переменных к аппаратным каналам (I/O Mapping) и структура ПО

Для связи программных экземпляров функциональных блоков (stateMachine, doorSM, ledSM), объявленных в глобальных переменных Global.var, с физическими входами и выходами стенда выполнена операция мэппинга (I/O Mapping). Все назначенные соответствия сведены в таблицу 5 и показаны на рисунках 13–15.

Таблица 5. Сводная таблица привязки программных переменных к аппаратным каналам модулей

Аппаратный модуль	Канал модуля	Программная переменная	Тип	Назначение канала
8I64xxxxxxx.00x-1 (ACOPOSinverter)	ETAD_Input	stateMachine.state	UINT	Слово состояния частотного преобразователя
8I64xxxxxxx.00x-1 (ACOPOSinverter)	CMDD_Output	stateMachine.command	UINT	Командное слово управления приводом
8I64xxxxxxx.00x-1 (ACOPOSinverter)	LFRD_Output	stateMachine.speed	INT	Заданная скорость вращения (об/мин)

Аппаратный модуль	Канал модуля	Программная переменная	Тип	Назначение канала
X20DI9371 (Дискретные входы)	DigitalInput01	doorSM.direction	BOOL	Тумблер направления (0 — откр., 1 — закр.)
X20DI9371 (Дискретные входы)	DigitalInput02	doorSM.sw1	BOOL	Датчик положения S0 (крайнее открытое)
X20DI9371 (Дискретные входы)	DigitalInput03	doorSM.sw2	BOOL	Датчик положения S1
X20DI9371 (Дискретные входы)	DigitalInput04	doorSM.sw3	BOOL	Датчик положения S2
X20DI9371 (Дискретные входы)	DigitalInput05	doorSM.sw4	BOOL	Датчик положения S3 (крайнее закрытое)
X20DO9322 (Дискретные выходы)	DigitalOutput02	ledSM.led1	BOOL	Светодиодный индикатор 1
X20DO9322 (Дискретные выходы)	DigitalOutput03	ledSM.led2	BOOL	Светодиодный индикатор 2
X20DO9322 (Дискретные выходы)	DigitalOutput04	ledSM.led3	BOOL	Светодиодный индикатор 3
X20DO9322 (Дискретные выходы)	DigitalOutput05	ledSM.led4	BOOL	Светодиодный индикатор 4

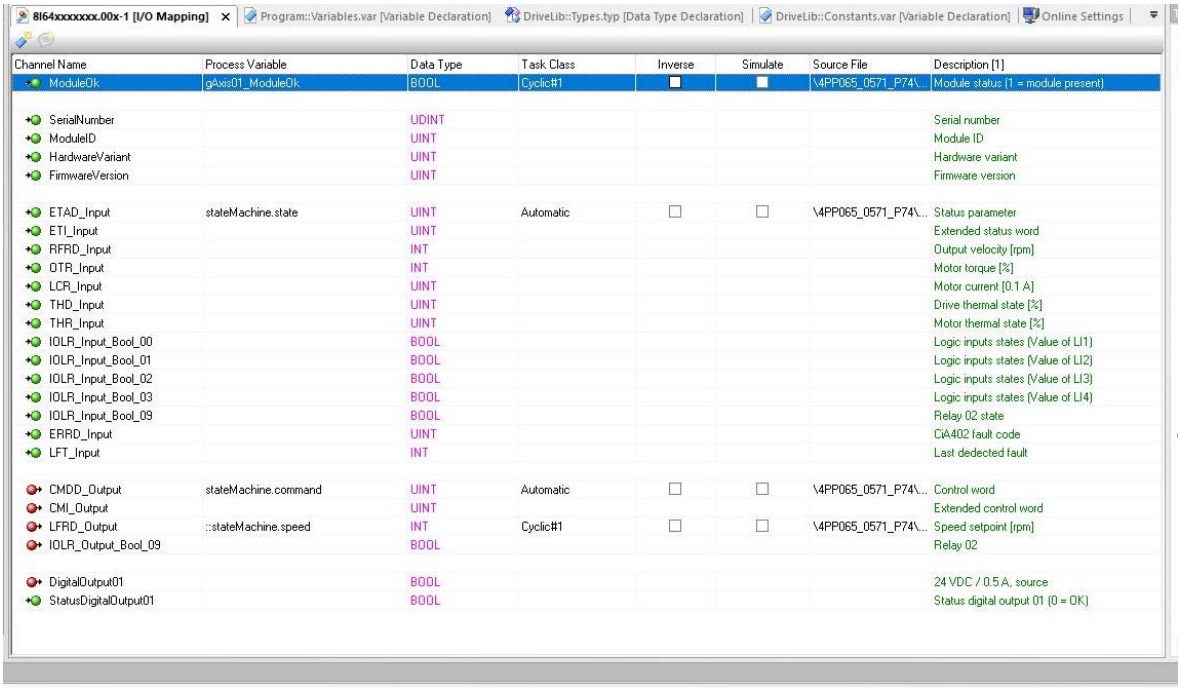
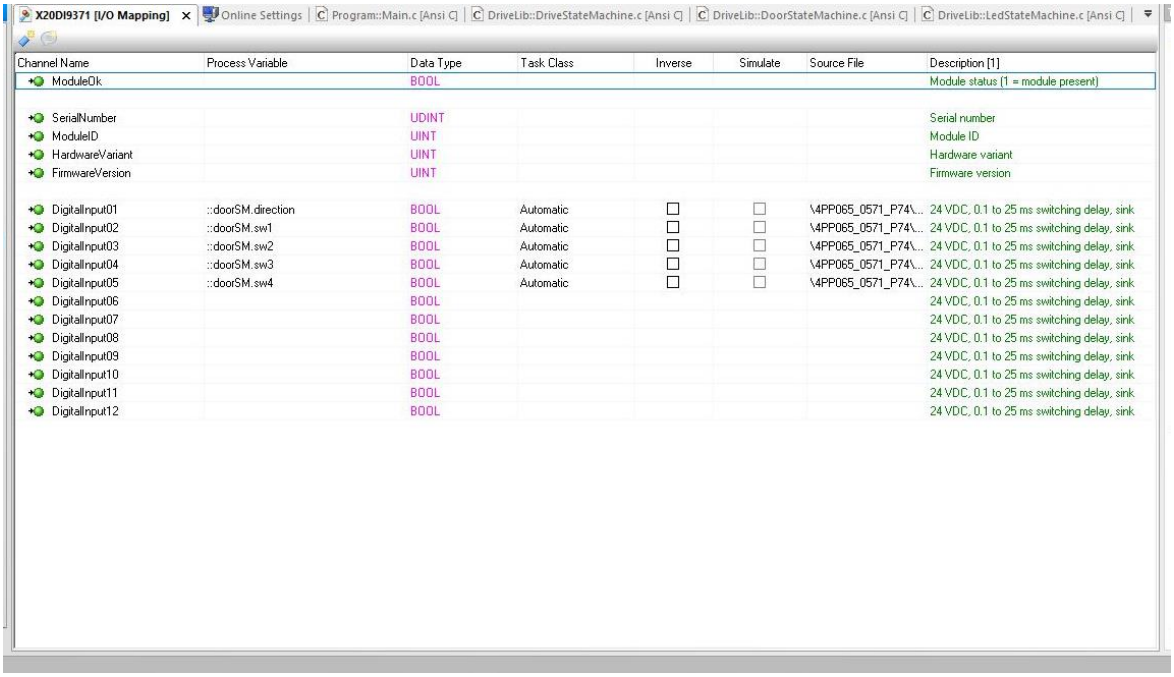
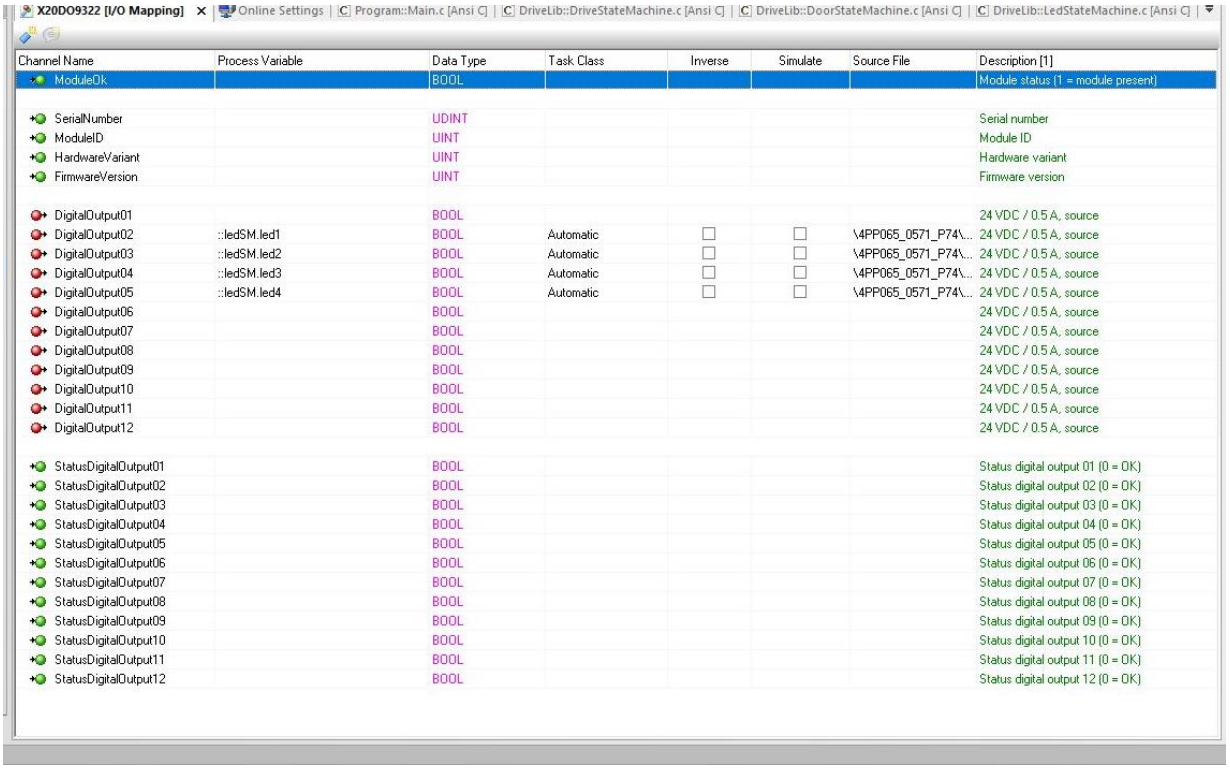


Рисунок 13 — Мэппинг каналов частотного преобразователя 8I64xxxxxxx.00x-1



Channel Name	Process Variable	Data Type	Task Class	Inverse	Simulate	Source File	Description [1]
ModuleOk		BOOL					Module status (1 = module present)
SerialNumber		UDINT					Serial number
ModuleID		UINT					Module ID
HardwareVariant		UINT					Hardware variant
FirmwareVersion		UINT					Firmware version
DigitalInput01	::doorSM.direction	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput02	::doorSM.sw1	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput03	::doorSM.sw2	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput04	::doorSM.sw3	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput05	::doorSM.sw4	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput06		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput07		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput08		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput09		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput10		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput11		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink
DigitalInput12		BOOL					24 VDC, 0.1 to 25 ms switching delay, sink

Рисунок 14. Мэппинг входных каналов модуля X20DI9371



Channel Name	Process Variable	Data Type	Task Class	Inverse	Simulate	Source File	Description [1]
ModuleOk		BOOL					Module status (1 = module present)
SerialNumber		UDINT					Serial number
ModuleID		UINT					Module ID
HardwareVariant		UINT					Hardware variant
FirmwareVersion		UINT					Firmware version
DigitalOutput01		BOOL					24 VDC / 0.5 A, source
DigitalOutput02	::ledSM.led1	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC / 0.5 A, source
DigitalOutput03	::ledSM.led2	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC / 0.5 A, source
DigitalOutput04	::ledSM.led3	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC / 0.5 A, source
DigitalOutput05	::ledSM.led4	BOOL	Automatic	☐	☐	\4PP065_0571_P74\...	24 VDC / 0.5 A, source
DigitalOutput06		BOOL					24 VDC / 0.5 A, source
DigitalOutput07		BOOL					24 VDC / 0.5 A, source
DigitalOutput08		BOOL					24 VDC / 0.5 A, source
DigitalOutput09		BOOL					24 VDC / 0.5 A, source
DigitalOutput10		BOOL					24 VDC / 0.5 A, source
DigitalOutput11		BOOL					24 VDC / 0.5 A, source
DigitalOutput12		BOOL					24 VDC / 0.5 A, source
StatusDigitalOutput01		BOOL					Status digital output 01 (0 = OK)
StatusDigitalOutput02		BOOL					Status digital output 02 (0 = OK)
StatusDigitalOutput03		BOOL					Status digital output 03 (0 = OK)
StatusDigitalOutput04		BOOL					Status digital output 04 (0 = OK)
StatusDigitalOutput05		BOOL					Status digital output 05 (0 = OK)
StatusDigitalOutput06		BOOL					Status digital output 06 (0 = OK)
StatusDigitalOutput07		BOOL					Status digital output 07 (0 = OK)
StatusDigitalOutput08		BOOL					Status digital output 08 (0 = OK)
StatusDigitalOutput09		BOOL					Status digital output 09 (0 = OK)
StatusDigitalOutput10		BOOL					Status digital output 10 (0 = OK)
StatusDigitalOutput11		BOOL					Status digital output 11 (0 = OK)
StatusDigitalOutput12		BOOL					Status digital output 12 (0 = OK)

Рисунок 15. Мэппинг выходных каналов модуля X20DO9322

Общая структурная схема информационного обмена между аппаратными модулями и функциональными блоками в задаче Program представлена на рисунке 16. В подпрограмме инициализации ProgramInit (_INIT) устанавливается разрешение работы инвертора stateMachine.enable = 1. В циклической части ProgramCyclic (_CYCLIC) на каждом шаге последовательно выполняются:

– инкремент счетчика тактов timeCounter и передача его значения в поле ledSM.timer;

– вызов блока DoorStateMachine(&doorSM), вычисляющего состояние двери и уставку скорости по датчикам и тумблеру;

– передача текущего состояния doorSM.state на вход ledSM.state и вызов блока индикации LedStateMachine(&ledSM);

– передача вычисленной скорости doorSM.speed в поле stateMachine.speed и вызов блока DriveStateMachine(&stateMachine), формирующего командное слово для инвертора.

5. Результаты экспериментальных исследований

Экспериментальная проверка разработанного ПО проводилась на стенде с помощью инструмента трассировки Trace, фиксирующего изменение переменных во времени. Поскольку в одной конфигурации Trace допускается запись не более 8 переменных одновременно, испытания проводились поэтапно: сначала исследовался режим начального поиска положения (ST_UNKNOWN), а затем — работа привода и светодиодной индикации на полном цикле закрытия и открытия двери.

Режим определения положения (ST_UNKNOWN). После подачи питания до срабатывания крайних датчиков автомат находится в состоянии ST_UNKNOWN (doorSM.state = 1). Как видно из графиков на рисунке 17, при положении тумблера direction = 0 привод вращается со скоростью −50 об/мин, при переключении тумблера в direction = 1 скорость изменяется на +50 об/мин, а при обратном переключении возвращается к −50 об/мин. Одновременно с этим все четыре светодиодных индикатора (ledSM.led1..led4) синхронно мигают с периодом 1,0 с (частота 1 Гц), что подтверждается диаграммой на рисунке 18.

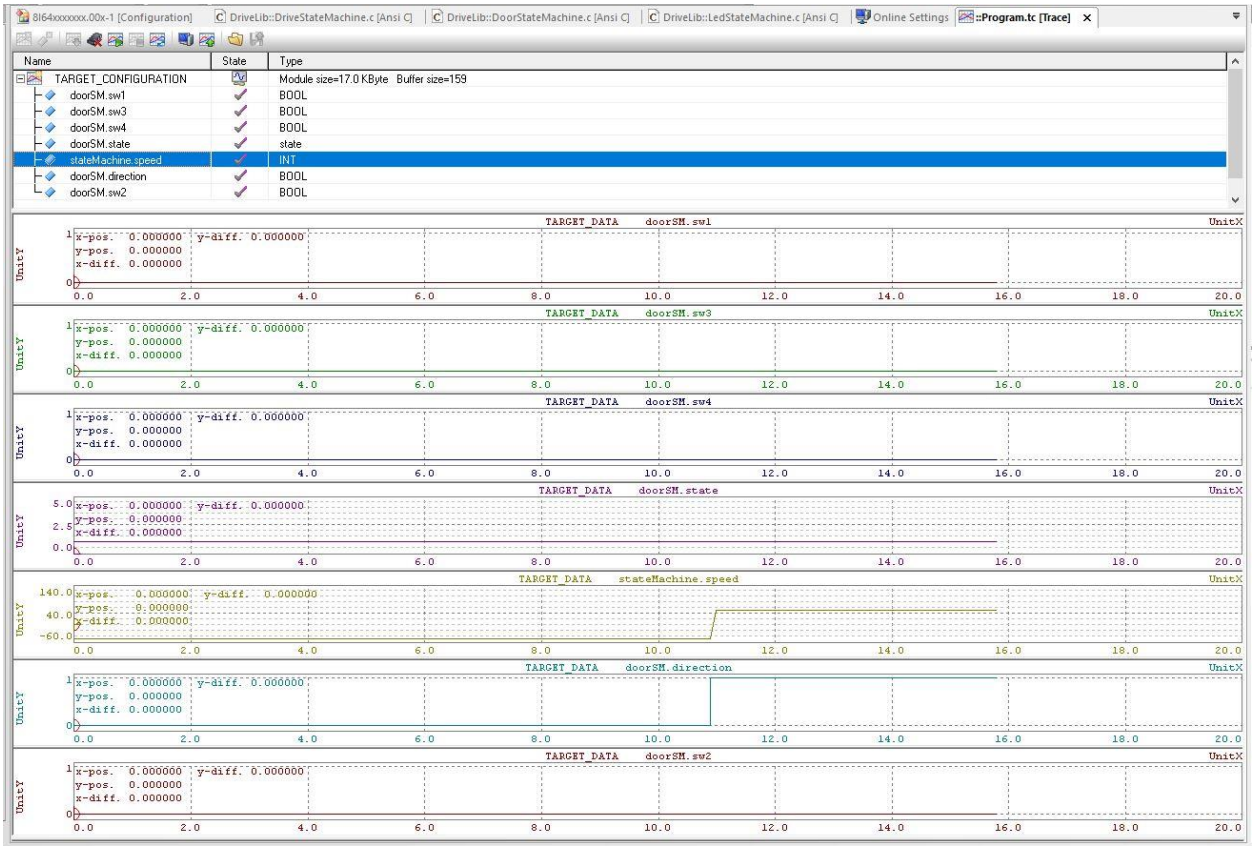


Рисунок 17. Диаграмма скорости привода и состояния автомата в режиме поиска положения (ST_UNKNOWN)

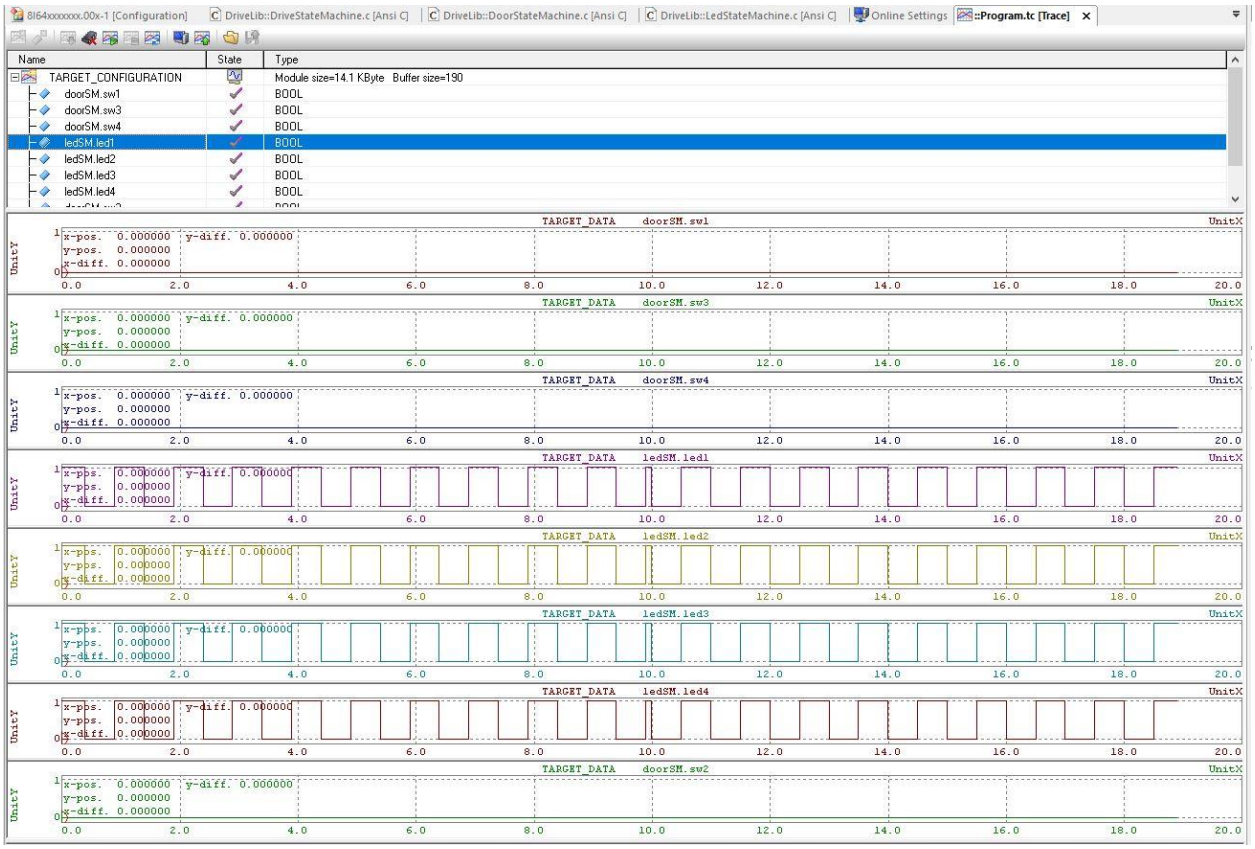


Рисунок 18. Синхронное мигание светодиодных индикаторов с частотой 1 Гц в режиме ST_UNKNOWN

Рабочий режим (полный цикл закрытия и открытия). После наезда двери на крайний датчик S0 (sw1 = 1 при direction = 0) система переходит в исходное открытое состояние ST_OPEN (state = 2, speed = 0, индикация 0000). Дальнейшая отработка команд на закрытие и открытие происходит в полном соответствии с алгоритмом (таблица 6). Временные диаграммы скорости, состояний автомата и датчиков за полный цикл приведены на рисунке 19, а диаграммы переключения светодиодных индикаторов — на рисунке 20.

Таблица 6. Последовательность отработки состояний, скорости и индикации в рабочем режиме

Направление хода	Событие / датчик	Переход в состояние (код)	Заданная скорость, об/мин	Код индикации (led1..4)
Ожидание (открыта)	Сработал S0 (sw1 = 1)	ST_OPEN (2)	0	0000
Закрытие (direction = 1)	Переключение тумблера в «1»	ST_ACC_POS (4)	+100	1000
Закрытие (direction = 1)	Прохождение датчика S1 (sw2 = 1)	ST_POS (6)	+200	1100
Закрытие (direction = 1)	Прохождение датчика S2 (sw3 = 1)	ST_DEC_POS (8)	+100	1110
Ожидание (закрыта)	Наезд на датчик S3 (sw4 = 1)	ST_CLOSE (3)	0	1111
Открытие (direction = 0)	Переключение тумблера в «0»	ST_ACC_NEG (5)	−100	1110
Открытие (direction = 0)	Прохождение датчика S2 (sw3 = 1)	ST_NEG (7)	−200	1100
Открытие (direction = 0)	Прохождение датчика S1 (sw2 = 1)	ST_DEC_NEG (9)	−100	1000
Ожидание (открыта)	Наезд на датчик S0 (sw1 = 1)	ST_OPEN (2)	0	0000

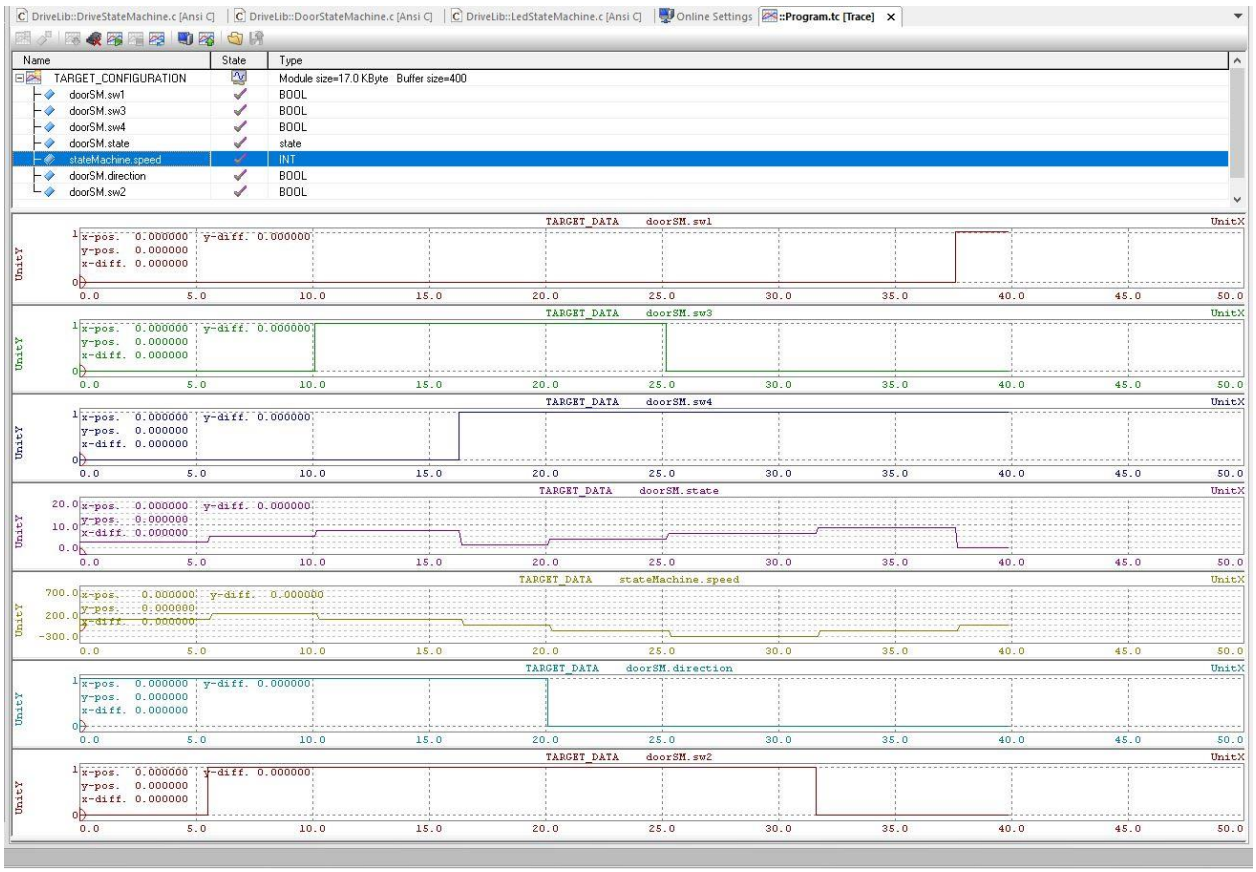


Рисунок 19. Графики изменения скорости привода и состояния автомата при полном цикле закрытия и открытия

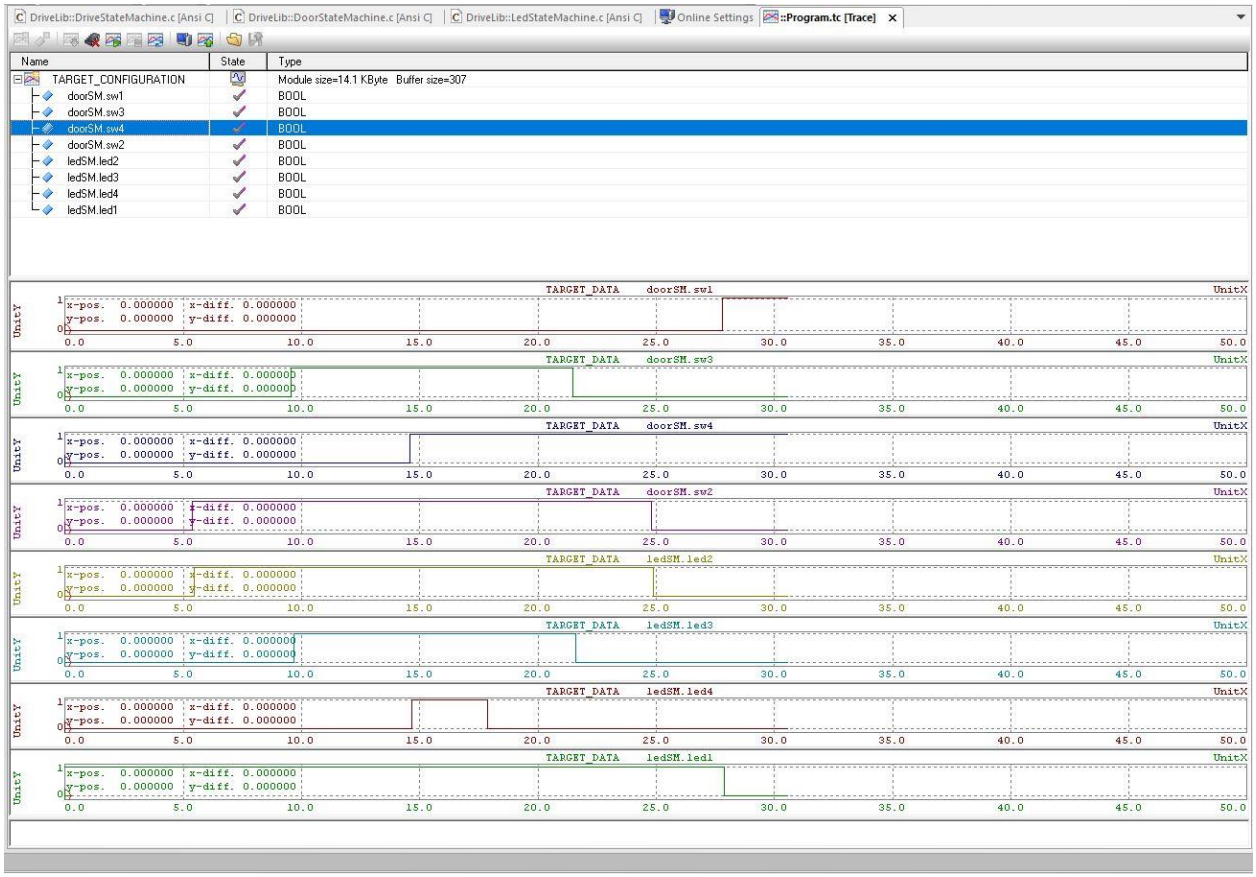


Рисунок 20. Диаграммы сигналов датчиков положения и светодиодных индикаторов за полный рабочий цикл

Анализ полученных графиков Trace показывает, что система строго выдерживает заданный трапецеидальный профиль скорости: разгон и движение вблизи крайних положений выполняются на безопасной скорости ±100 об/мин, максимальная скорость ±200 об/мин включается только на центральном участке S1–S2, а светодиодная линейка безошибочно отражает текущую координату двери.

Выводы

В ходе выполнения лабораторной работы в среде B&R Automation Studio было разработано и экспериментально протестировано программное обеспечение системы автоматического управления электроприводом защитной двери на базе функциональных блоков и конечных автоматов.

При выполнении работы решены следующие инженерные задачи:

– сконфигурирована аппаратная часть учебного стенда на базе контроллера 4PP065_0571_P74, подключен по шине X2X частотный преобразователь ACOPOSinverter 8I64xxxxxxx.00x-1, заданы паспортные параметры асинхронного электродвигателя, а регистры управления и состояния переведены в упакованный формат (packed);

– создана пользовательская библиотека DriveLib на языке ANSI C, инкапсулирующая логику трех конечных автоматов в отдельных функциональных блоках: DriveStateMachine (управление готовностью инвертора), DoorStateMachine (траекторно-логический автомат двери из 10 состояний) и LedStateMachine (управление индикацией и генерация меандра 1 Гц);

– выполнена привязка (I/O Mapping) программных переменных к физическим каналам инвертора и модулей дискретного ввода-вывода X20DI9371 и X20DO9322;

– сняты экспериментальные диаграммы средством Trace, полностью подтвердившие работоспособность системы как в режиме начального поиска

положения на малой скорости (±50 об/мин с миганием индикаторов 1 Гц), так и в штатном рабочем режиме с трехступенчатым профилем скорости (±100 / ±200 / ±100 об/мин) и возможностью безударного реверса на любом участке пути.

Приложение А – листинг программного кода

A.1. Main.c — основная программа

```
#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif

void _INIT ProgramInit(void)
{
    stateMachine.enable = 1;
}

void _CYCLIC ProgramCyclic(void)
{
    timeCounter++;
    ledSM.timer = timeCounter;
    DoorStateMachine(&doorSM);
    ledSM.state = doorSM.state;
    LedStateMachine(&ledSM);
    stateMachine.speed = doorSM.speed;
    DriveStateMachine(&stateMachine);
}

void _EXIT ProgramExit(void)
{

}
```

A.2. Global.var — глобальные переменные

```
VAR
    gAxis01 : ACP10AXIS_typ;
    gAxis01_HW : SdcHwCfg_typ := (0);
    gAxis01_DrvIf : SdcDrvIf16_typ := (0);
    gAxis01_DiDoIf : SdcDiDoIf_typ := (0);
    gAxis01_ModuleOk : BOOL := 0;
    doorSM : DoorStateMachine;
    ledSM : LedStateMachine;
    stateMachine : DriveStateMachine;
END_VAR
```

A.3. Variables.var — локальные переменные программы

```
VAR
    timeCounter : USINT := 0;
END_VAR
```

A.4. DriveLib.fun — объявление функциональных блоков

```
{REDUND_ERROR} FUNCTION_BLOCK LedStateMachine (*TODO: Add your comment here*)
(*$GROUP=User,$CAT=User,$GROUPICON=User.png,$CATICON=User.png*)
    VAR_INPUT
        state : state := ST_INIT;
    END_VAR
    VAR_OUTPUT
        led1 : BOOL;
        led2 : BOOL;
        led3 : BOOL;
        led4 : BOOL;
        timer : INT;
    END_VAR
END_FUNCTION_BLOCK
```

```
{REDUND_ERROR} FUNCTION_BLOCK DriveStateMachine (*TODO: Add your comment here*)
(*$GROUP=User,$CAT=User,$GROUPICON=User.png,$CATICON=User.png*)
    VAR_INPUT
        state : UINT;
        enable : BOOL;
    END_VAR
    VAR_OUTPUT
        command : UINT;
        speed : INT;
    END_VAR
END_FUNCTION_BLOCK


{REDUND_ERROR} FUNCTION_BLOCK DoorStateMachine (*TODO: Add your comment here*)
(*$GROUP=User,$CAT=User,$GROUPICON=User.png,$CATICON=User.png*)
    VAR_OUTPUT
        state : state := ST_INIT;
    END_VAR
    VAR_INPUT
        sw1 : BOOL;
        sw2 : BOOL;
        sw3 : BOOL;
        sw4 : BOOL;
        direction : BOOL;
    END_VAR
    VAR_OUTPUT
        speed : INT;
    END_VAR
END_FUNCTION_BLOCK
```

A.5. Types.typ — перечисление состояний двери

```
TYPE
    state :
        (
        ST_DEC_NEG := 9,
        ST_DEC_POS := 8,
        ST_NEG := 7,
        ST_POS := 6,
        ST_ACC_NEG := 5,
        ST_ACC_POS := 4,
        ST_CLOSE := 3,
        ST_OPEN := 2,
        ST_UNKNOWN := 1,
        ST_INIT := 0
        );
END_TYPE
```

A.6. Constants.var — константы

```
VAR CONSTANT
    CMD_ENABLE : UINT := 15;
    CMD_SHUTDOWN : UINT := 6;
    CMD_SWITCH_ON : UINT := 7;
    STATE_DISABLED : UINT := 64;
    STATE_READY : UINT := 33;
    STATE_SWITCHED_ON : UINT := 35;
END_VAR
```

A.7. DriveStateMachine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "DriveLib.h"
#ifdef __cplusplus
    };
#endif
```

```
/* TODO: Add your comment here */
void DriveStateMachine(struct DriveStateMachine* inst)
{
    /*TODO: Add your code here*/
    UINT mask = inst->state & 0x6f;
    if(!inst->enable){
        inst->command = CMD_SHUTDOWN;
    }
    else{
        switch (mask){
            case STATE_DISABLED:
                inst->command = CMD_SHUTDOWN;
                break;

            case STATE_READY:
                inst->command = CMD_ENABLE;
                break;

            case STATE_SWITCHED_ON:
                inst->command = CMD_SWITCH_ON;
                break;
        }
    }
}
```

A.8. DoorStateMachine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
#include "DriveLib.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void DoorStateMachine(struct DoorStateMachine* inst)
{
    //
    switch (inst->state){
        /*
            Для согласованности индикации, тумблера, отвечающего за направление, и кнопок,
        отвечающих за датчики, направление ворот, в отличие от методички, выбрано так, что
        открытое состояние, когда ворота слева, и закрытое - справа. Т.о. direction == 0
        обозначает направление влево(открытие), а direction == 1 - вправо(закрытие)
        */
        case ST_INIT:
            inst->speed = 0;
            inst->state=ST_UNKNOWN;
            break;

        case ST_UNKNOWN:
            /*
                Из неизвестного состяния мы можем перейти в одно из двух, открыты или закрыты.
            При этом помимо направления, также должен быть задействован соответствующий датчик,
            иначе остаемся в неизвестном состоянии
            */
            if (inst->sw1 && inst->direction == 0){
                inst->state = ST_OPEN;
                break;
            } else if (inst->sw4 && inst->direction == 1){
                inst->state = ST_CLOSE;
                break;
            } else {
                inst->speed = (inst->direction == 0)?-50:50;
                break;
            }

        case ST_OPEN:
            /*
                Если ворота открыты, а направление остаётся тем же (влево), то скорость остаётся
нулевой, остаёмся в том же состоянии.
            Иначе, если направление теперь другое (вправо) то переход в состояние ACC_POS
(узкорение в сторону закрытия (вправо).
```

```
            В случае неизвестной комбинации сигналов с датчиков - отправка в состояние UNKNOWN.
            */
            // Остаемся в ST_OPEN
            if (inst->direction == 0){
                inst->speed = 0;
                break;
            }
            // Переход в ST_ACC_POS
            else if (inst->direction == 1){
                inst->state = ST_ACC_POS;
                break;
            }
            // Переход в UNKNOWN
            else {
                inst->state = ST_UNKNOWN;
                break;
            }

        case ST_ACC_POS:
            /*
                Если ворота ускоряются, то они остюаются в том же положении пока не сработал
второй датчик
            или не изменилось направление. Когда сработал второй датчик, то устанавливается
максимальная
            скорость и состояние ST_POS. Когда меняется направление, то устанавливается обратная
скорость
            и состояние ST_DEC_NEG
            */
            // Остаемся в ST_ACC_POS
            if (inst->direction == 1 && !inst->sw2){
                inst->speed = 100;
                break;
            }
            // Сработал второй датчик, переход в ST_POS
            else if (inst->direction == 1 && inst->sw2){
                inst->state = ST_POS;
                break;
            }
            // Изменилось направление, переход в ST_DEC_NEG, установка обратной скорости
            else if (inst->direction == 0) {
                inst->state = ST_DEC_NEG;
                break;
            }
            // Неизвестное состояние
            else {
                inst->speed = 0;
                inst->state = ST_UNKNOWN;
                break;
            }

        case ST_POS:
            /*
                Состояние максимальной скорости, движение в сторону закрытия. Остаёмся в том же
состоянии пока не сработал третий датчик.
            После срабатывания третьего датчика переход в ST_DEC_POS. Если сменится направление
то переход в ST_NEG.
            */
            // То же состояние
            if (inst->direction == 1 && !inst->sw3){
                inst->speed = 200;
                break;
            }
            // Сработал 3 датчик, замедление
            else if (inst->direction == 1){
                inst->state = ST_DEC_POS;
                break;
            }
            // Смена направления
            else if (inst->direction == 0){
                inst->state = ST_NEG;
                break;
            }
            // Неизвестное состояние
            else{
                inst->state = ST_UNKNOWN;
                break;
            }
```

```
        case ST_DEC_POS:
            /*
                Состояние замедления перед закрытием. При отстутсвии новых сигналов остаётся в
том же состоянии. При наличии сигнала с последнего
            датчика переход в состояние ST_CLOSE. При смене направления, переход в состояние
ST_ACC_NEG. Иначе ST_UNKNOWN.
            */
            // Сохранение состояния
            if (inst->direction == 1 && !inst->sw4){
                inst->speed=100;
                break;
            }
            // Переход в закрытое состояние
            else if (inst->direction == 1 && inst->sw4)
            {
                inst->state = ST_CLOSE;
                break;
            }
            // Смена направления
            else if (inst->direction == 0)
            {
                inst->state = ST_ACC_NEG;
                break;
            }
            // Переход в неизвестное состояние
            else{
                inst->state = ST_UNKNOWN;
                break;
            }

        case ST_CLOSE:
            if (inst->direction == 1)
            {
                inst->speed = 0;
                break;
            }
            else if(inst->direction == 0){
                inst->state = ST_ACC_NEG;
                break;
            }
            else
            {
                inst->state = ST_UNKNOWN;
                break;
            }

        case ST_ACC_NEG:
            /*
                Состояние открывающихся ворот, сохраняет состояние пока не сработает 3 датчик или
не изменится направление
            */
            if (inst->direction == 0 && !inst->sw3){
                inst->speed = -100;
                break;
            }
            else if (inst->sw3){
                inst->state = ST_NEG;
                break;
            }
            else if (inst->direction == 1){
                inst->state = ST_DEC_POS;
                break;
            }
            else {
                inst->state = ST_UNKNOWN;
                break;
            }

        case ST_NEG:
            /*Открытие ворот на макс скорости*/
            if (inst->direction == 0 && !inst->sw2){
                inst->speed = -200;
                break;
            }
            else if (inst->direction == 0) {
                inst->state = ST_DEC_NEG;
                break;
            }
```

```
            else if (inst->direction == 1){
                inst->state = ST_POS;
                break;
            }
            else{
                inst->state = ST_UNKNOWN;
                break;
            }
        case ST_DEC_NEG:
            if (inst->direction == 0 && !inst->sw1){
                inst->speed = -100;
                break;
            }
            else if(inst->direction == 0){
                inst->state = ST_OPEN;
                break;
            }
            else if (inst->direction == 1){
                inst->state = ST_ACC_POS;
                break;
            }
            else {
                inst->state = ST_UNKNOWN;
                break;
            }

    }
}
```

A.9. LedStateMachine.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "DriveLib.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void LedStateMachine(struct LedStateMachine* inst)
{
    switch(inst->state){
        case ST_INIT:
            inst->led1 = 0;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;

        case ST_UNKNOWN:
            if (inst->timer%5==0){
                inst->led1=!inst->led1;
                inst->led2=!inst->led2;
                inst->led3=!inst->led3;
                inst->led4=!inst->led4;
            }
            break;

        case ST_OPEN:
            inst->led1 = 0;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;

        case ST_ACC_POS:
            inst->led1 = 1;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
```

```
        case ST_POS:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 0;
            inst->led4 = 0;
            break;

        case ST_DEC_POS:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 1;
            inst->led4 = 0;
            break;

        case ST_CLOSE:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 1;
            inst->led4 = 1;
            break;

        case ST_ACC_NEG:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 1;
            inst->led4 = 0;
            break;

        case ST_NEG:
            inst->led1 = 1;
            inst->led2 = 1;
            inst->led3 = 0;
            inst->led4 = 0;
            break;

        case ST_DEC_NEG:
            inst->led1 = 1;
            inst->led2 = 0;
            inst->led3 = 0;
            inst->led4 = 0;
            break;
    }
}
```